

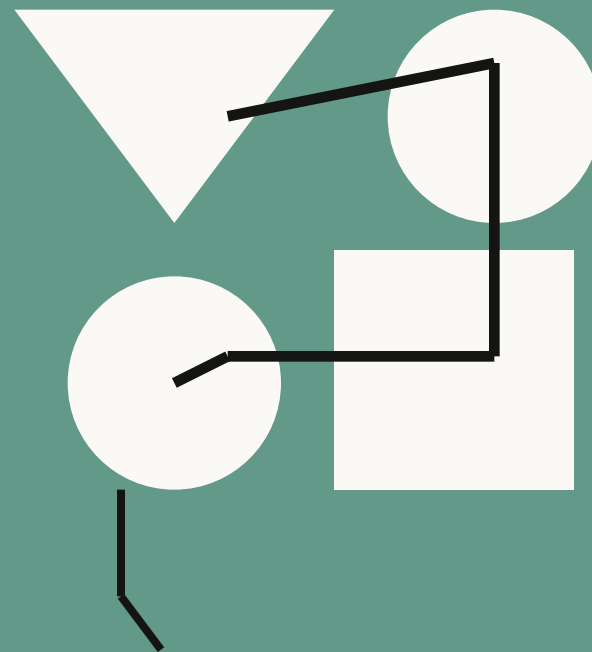
创始人手册： 构建 AI 原生 创业公司

目录

2026 年重启的创业生命周期	3
创始人意味着什么，正在改变	5
Idea 阶段：构想期	8
MVP 阶段：最小可行产品	15
Launch 阶段：上线期	21
Scale 阶段：扩张期	25
同样的工作，新的规则	31
延伸资源	33

Chapter 1

2026 年重启的 创业生命周期



2026 年重启的创业生命周期

AI 正在重塑创业公司的构建方式。今天，从未写过一行代码的创始人也能让生产级应用上线，而“十人独角兽”已经从草根逆袭的传说变成可被刻意规划的路径。

到 2026 年，AI 可以编写生产代码、做市场调研、综合竞争格局、起草投资人材料、并自动化日常运营流程。它把过去让连资深技术创始人都头痛的工具集成、平台对接、系统打通的学习曲线削平，让“谁可以创办公司”或“谁可以做出产品”这件事，第一次真正实现了起点的平权。

2026 年，一个好的想法能把创始人带得比以往任何时候都远。Agentic coding 把过去需要一整支工程团队完成的工作，压缩成创始人一个人就能交付的体量。

传统的创业增长曲线默认路径是：验证 → 融资 → 招人 → 开发 → 再融资 → 增长 → 招更多人 → 循环。如今，AI 抹掉了“每进入新阶段就必须更大团队、更新技能矩阵、更多融资”的预期。

本手册按照这些新现实重新绘制了创业旅程的四个核心阶段（Idea、MVP、Launch、Scale）：当 AI 成为你技术与组织发展的核心基础设施时，每个阶段会是什么样子？该用哪些工具？正在使用这些工具的创始人，又是如何压缩时间线的？

如果你准备好把“从 Idea 到 Exit”的路径走到最短，请继续读下去。

Chapter 2

创始人意味着什么， 正在改变

创始人意味着什么，正在改变

创始人过去被“能做什么”来定义：技术创始人写代码，非技术创始人跑业务、谈合同。但 2026 年的模型、系统和 AI 智能体已经溶解了“能造东西的人”与“有值得造的想法的人”之间的那堵墙。

AI 原生创业公司正在根本性地重塑“创始人”的含义。没有工程背景的人也能做出生级软件让自己的想法落地；技术过硬但商业经验有限的创始人，也能轻松产出 GTM 策略、财务模型与精打细磨的融资 PPT。

过去，创始人大量时间花在执行模式：写代码、管人、处理日常运营。在 AI 原生创业公司里，创始人角色正从“一线工人”变成“智能体编排者”——你在调度一组能读文件、跑命令、执行代码、甚至浏览网页的专业 AI 助手。注意力随之上移，转向更高一层的工作：产生想法，并引导承载这些想法的系统去落地。

把 AI 作为核心基础设施带来的最革命性结果，是它解放了“有领域专长但没有工程背景”的创始人。当创始人池扩大到工程师之外，你会看到由完全不同生命经验的人创办的公司，去解决传统科技创始人通道从未优先关注（甚至从未注意）的真问题。

适合精益创业公司的 AI 能力

传统模型假设你必须雇工程师做产品、销售去卖、运营去维持公司，员工数被视为组织势能与产品成熟度的标志。

2026 年早期创业公司截然不同：在设计上极度精益，往往就是创始人一人或加几位同伴。把技术与组织都建立在 AI 之上，他们能在不扩大团队的情况下达成产品验证、初步收入甚至盈利。

AI 在三个方面尤其能让一家公司表现得像一家更大组织：研究、agentic coding、与关键业务运营的工作流自动化。

对话式智能与研究

想象：随叫随到、跨任何领域的专家

第一年里创始人几乎都不懂但又必须懂的事很多：怎么搭薪资？怎么规划产品 sprint？怎么写紧凑的投资人备忘录？过去答案永远是“找个懂的人”——要么花掉本该 building 的时间，要么烧掉早期资本请顾问。现在，他们随时拥有跨任何领域的 on-call 专家。

- 深度研究：竞品分析、市场规模、财务建模
- 文档起草：融资 PPT、案例研究、投资人备忘录、PRD
- 战略思考伙伴：唱反调分析、pre-mortem、情景推演、路线图优化

Agentic Coding（代理式编程）

想象：永远在线、永不被阻塞的工程师

过去，做软件要么找一位技术联创，要么外包给一家开发公司，要么把跑道烧够长以便先招够工程团队再写代码。

Agentic coding 工具让每一位胸怀想法的创始人都能用自然语言描述自己要造的东西，由 AI 以整个工程团队的速度与体量生成、测试、调试、重构生产级代码库。“我有一个想法”到“我有一个产品”的时间线被显著压缩。创始人的角色重心转向决定造什么、为什么造，AI 负责真正搭起可面对真实用户的基础设施。

workflow 自动化

想象：随叫随到的自动化运营团队

即便创始人能像顾问一样做研究、像工程团队一样写代码，仍有一大类工作必须有人做：排日程、更新 CRM、出周报、维护文档、发内容、跟踪合规、把公司运转所依赖的工具与系统串起来。在精益创业公司里，这副担子主要落在创始人身上——是一笔严重透支高阶决策时间与注意力的“税”。

AI 工具加持的工作流自动化能替你交这笔税。重复性运营任务可以配置成自动发生：deal 状态变更时 CRM 自动更新，周报自动汇编，产品文档随产品变更同步更新。关键是，Claude Cowork 能集成创业公司赖以运行的互联系统——项目管理工具、沟通栈、数据源——你不需要专门有人去搭建并维护这些 integrations。而在 Day-Zero 创业公司里，那个“专门的人”几乎总是创始人本人。

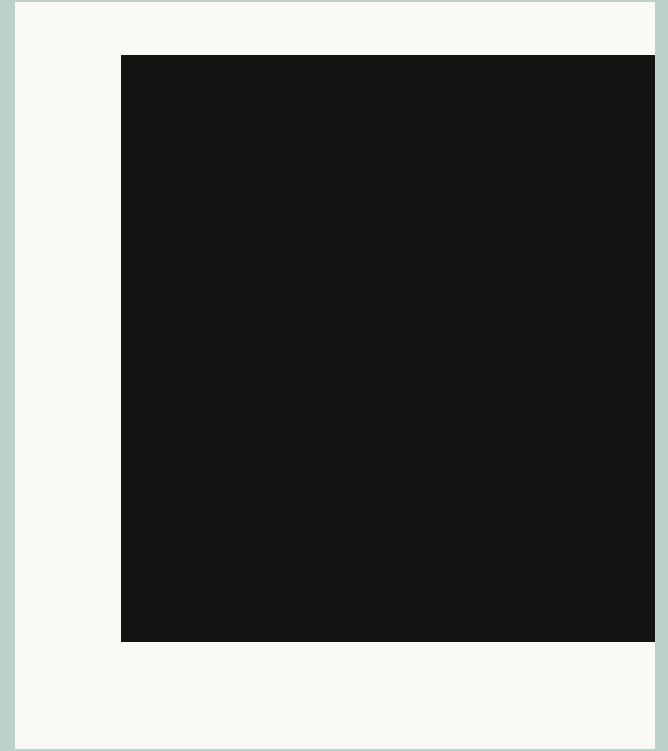
时机与编排，决定一切

能够有效驾驭 AI 的研究、自动化与 agentic coding 能力的创始人，可以让一家公司以远超其人头数的杠杆运转，也能把绝大多数时间与带宽留给真正重要的工作。

但这不会“自动驾驶”——执掌这些 AI 工具的创始人需要知道在何时、如何使用它们。本手册的其余部分，将探讨创始人在 AI 原生创业之旅每个阶段会遭遇的目标与挑战，以及如何在每个阶段有效应用 AI 工具。

Chapter 3

Idea 阶段 (构想期)



Idea 阶段：构想期

每个创业者都从同一个地方出发：一个让自己念念不忘的问题。Idea 阶段是想法第一次与现实碰头的阶段——2026 年要想创业成功，必须有一种“在证据足以支撑之前不开始 Build”的纪律。本阶段的工作是研究、客户访谈、竞品分析，以及对反对你的证据的诚实评估——这些都要在你让 Claude Code 生成第一行生产代码之前完成。

Idea 阶段的目标

本阶段创始人的核心目标是研究导向的验证：在投入资源建造之前，先收集到充分证据，证明（1）真问题确实存在，（2）你设想的解决方案能真正解决这个问题。

实际上，Idea 阶段是按下面顺序逐一回答的问题：

- 这个问题是否足够真实、具体、高频，足以据此建一家公司？
- 究竟谁拥有这个问题？他们能构成一个市场吗？
- 是否已有人在解决？如果有，他们怎么做、做得多好？
- 一个解决方案需要做到哪些事，才能真正解决这个问题——你的 idea 做到了吗？

这些问题最终汇聚为一个终极问题：这件事值得 Build 吗？

“人们对报销很头疼”是一句观察；“中型公司的财务经理每周花 4 小时以上对账，因为现有工具与会计软件不能集成”——这才是可被检验的假设。

退出标准

退出条件是：找到 problem-solution fit。在着手做“那个解决方案”之前，已经主要通过真实人类对话拿到了定性证据，证明你确实是在为真实存在的人解决一个真实存在的问题。

当你能对下列三问都坚定回答“是”时，可以离开 Idea 阶段：

1. 问题是否真实且具体？能精确说出：谁会遇到、多频繁、多严重、目前如何应对。
2. 你的方案是否解决了真正的问题？不是你最初臆想的，而是验证过程中浮现的那个。
3. 信号是否足以支撑开始建造？永远不会有 100% 确定，但需要足够定性证据，让“投入 MVP”成为推理决策。

本阶段挑战

Idea 阶段是整个旅程里最关键的工作，也是最大错误的发生地：判断错了会让萌芽中的项目迅速脱轨。本阶段多数挑战都源于“走得比理解更快”——稳重审慎、有节奏推进的创始人能稳步前行。

把“Build”误认成“验证”

挑战：技术门槛一旦移除，热血上头的创始人最容易跳过创业旅程中最关键的一步——验证 idea 是否真是用户需要并会用的方案。

哪怕在 agentic coding 流行起来之前，42% 的创业公司失败是因为做出了无人想要的东西。Claude Code 等工具又戏剧性地压缩了“我有 idea”到“我有产品”之间的距离，这个失败率只会上升。

对一个有“突触为之一震”般好想法的人来说，没有比现在更好的当创始人的时机。但与此同时，原型也以前所未有的速度被搭出来——这反而对 AI 原生创业公司构成了真实的存亡风险。

通往 problem-solution fit 的正确路径是：先验证假设、再 Build。但许多创始人误以为 AI 让这一步可“短路”，于是变成：有 idea → 立刻搭原型 → 把原型的存在当作验证。原型反过来成为“我的假设一开始就对”的理由——而那个假设从未被真正测试过。

一个能跑起来的原型很容易被误当作“在解决真实问题”的证据，但它不是。原型更恰当的角色是与潜在用户对话时的压力测试道具。真正的证据，是这些对话本身。

过早扩张

挑战：当 Build 几乎不费力气且即时可得时，你可以在业务还未要求之前，就把执行规模冲到太远。

过早扩张的意思是：在你尚未真正验证某条产品路径值得 commit 之前就 commit 了它。这一直是创业杀手，AI 时代让创始人更容易不知不觉踩进这个陷阱。

AI 会以同等热情，围绕一个本质上有缺陷的前提，去生成、测试、调试、重构整个代码库。系统中的 intelligence 是你的判断力。最高准则：让你的“理解”始终走在你的“建造”之前——尤其当建造如此轻快之时。

客观性的丢失

挑战：让 AI 工具去找支持你已经相信的事的证据，它一定能找到。确认偏误如今配上了研究引擎。

AI 跟随你的指令——所以不去问硬问题的创始人，能比以往更快为一个糟糕的 idea 构造出“看起来研究充分”的精致论证。解药是同一个工具，反向使用：让 AI 像验证一个 idea 那样彻底地压力测试它。当研究与结构化对抗性思考浮现出“需要修订”的证据时，那正是 pivot 的信号。

Claude 如何帮助 Idea 阶段的创始人

Idea 阶段往前推进常常感觉旷日持久。你是一个创始人，你只想开始 Build。但这一关键启动期本质上是研究与验证练习——意味着在全力写代码之前，要先伸手去拿能让你思考更严谨的工具。下面是在 Claude 的三个产品形态（Chat、Claude Cowork、Claude Code）上完成 Idea 阶段、并在尽职调查上不打折扣的方式。

定义并压力测试问题假设

你的领域专长与前期研究已经产生了一个假设。第一工作是把它锐化到真正可被检验。Claude 在这里特别擅长强迫具体化：究竟谁有这个问题？多频繁？多严重？目前如何应对？无法精确回答这些问题的问题陈述，不具备验证条件。

Exercise： 与 Claude 一起把问题陈述磨成可检验的假设。“合同审查太久了”基本无法被验证；但“中型公司的内部法务团队，平均每个合同审查周期要花 3 天以上，因为修订意见分散在邮件里而非单一受版本控制的文档”，就极具可验证性。

接下来让 Claude 反过来反驳你的 idea，主动去找能证伪你假设的证据。这能浮现负面市场信号、失败竞品、用户行为模式与结构性障碍——一个支持性合成会悄悄忽略掉的那些。

选对 Claude 形态

AI 让创始人能更快出货、更高效自动化、放大规模——但你选哪个形态来用，差别很大。

Chat

在你正在使用的 app 里做快速交流。日常小任务：从一份长篇投资备忘录中提取一句结论，会前快速核实一个数据，或把一条冗长 Slack 讨论整理清楚。

Claude Cowork

处理真正需要时间的知识工作。从多个来源拉信息、综合，并产出成品（文档、PPT、表格）。比如把一堆客户访谈录音转写整理成主题发现文档、融资前从十几个 vendor 网站搭出竞争格局，或周一固定从已连接的工具中抓指标，生成本周 KPI 简报放进共享文件夹。

Claude Code

给团队工程师的 agentic coding 环境。直接访问代码库，Plan Mode，git 集成，本地 / IDE / 沙盒云端环境。一支精益团队由此能在不断长大的代码库里持续出货、把 MVP 时期的遗留代码迁移、从原型推进到生产——无需先招更多人。

三者共享同一个底层 Claude，变化的是它周围的工作空间。

目标是：到客户访谈之前，已经先用最强反方论点把假设过了一遍。这样后续的探索性用户访谈就是真正开放的，而不是为印证而搜寻。

Note：在 AI 创业全生命周期里，把 Claude 用作结构化的“魔鬼代言人”是一项核心场景。

市场研究与竞争格局映射

盘点竞争对手

创业公司里有一个特定现象叫做“竞争忽视”：过度专注于自己的愿景与执行，从而系统性低估同空间内别人在做的事。AI 提供了解药：让 Claude 做出“为什么这个方向上的竞争对手会成功而你失败”最有说服力的论证。

Claude 会分析他们的方案为何更优，客户为何选他们，你以为是差异点为何没那么牢。

Exercise：让 Claude 按层级映射你的竞争格局：直接竞争者、间接竞争者、潜在收购者、可能切入此领域的相邻玩家。然后让它论证每一层为何真的构成威胁，而不是最容易被打发的那个版本。

市场研究

Claude Code 可以汇总公开的客户反馈，浮现反复出现的抱怨与未满足的需求。附加价值：这相当于免费做了一遍对手客户的定性研究。

Exercise：让 Claude Cowork 综合关键来源的竞品评论，识别现有方案尚未解决的 Top 抱怨。如果你的假设命中其中一个或多个，那是 problem-solution fit 的强信号；如果没有，那也值得知道。

Claude Cowork 还可以从密集的行业报告、分析师备案、市场研究文档中提取关键信息和数字——这些被清洗、合成后的输入，正适合作 Claude 分析的上下文。

Exercise：用公开数据搭出 TAM / SAM / SOM 模型，并对背后假设做压力测试。判断市场是在扩张、收敛还是成熟——这会影响你怎么看待时机与差异化。再画一遍买方版图：谁掌握预算？谁影响决策？两者是否同一个人？

趋势分析

用 Claude 收听早期指标，判断你是否在合适的时机进入：跟踪那些已经在讨论这个问题的 subreddit 与 LinkedIn 群组，留意用户在描述自己困境时实际选用的具体词汇。让 Claude 识别曾经解决过类似问题的相邻市场，提炼哪些方法奏效、哪些不奏效。让监管、技术或人口结构的趋势浮出水面——它们可能加速或威胁这个机会。

Exercise：让 Claude 找出未来两年可能显著影响你市场的三个外部趋势（监管、技术或人口结构），并评估每一个对你的具体假设而言是顺风还是逆风。

Note：本节的市场研究与竞争映射不是一次性练习。你会在 MVP 与 Launch 阶段持续发现新东西、不断更新认知，所以每当假设演变时，都应重做一次。

规划并设计客户访谈

你从潜在用户身上学到的东西的质量，取决于：(1) 提问的质量；(2) 你是否在向正确的人提问。Claude 尤其擅长协助客户访谈——谁去谈、谈什么、如何理解听到的内容。

找谁谈

一个精确的目标画像，远比一份长长的联系人清单更值钱：具体的职位、公司类型、团队结构、最可能尖锐感受到此问题的层级。然后弄清这些人哪里能被触达——他们聚集的社区、活动、LinkedIn 群组、Slack 空间——并按“离问题距离”建立优先级框架。

问什么

目标确定后，用 Claude 搭出访谈框架本身：合适的问题、合适的顺序，结构上能让对方说出“实际做了什么”，而不是“自以为会做什么”。新手错误是问一个泛而开放的未来问题（“你会用这种产品吗？”），而不是具体询问相关的过去（“跟我讲讲你上次处理这个问题的过程”）。

Claude 也会标出你草拟的问题里哪些有引导性、太宽泛、或大概率产出噪声而非信号；还能为可能的回避或含糊回答设计追问。如果你的假设涉及不只一种 persona，Claude 还能为每种 persona 设计不同的问卷。

Exercise： 先自己手写访谈问题，再让 Claude 审核：明确指示它标出有引导性、面向未来、太宽泛、或更容易产出“社交期许”答案的问题；再请它针对最容易出现回避的两三个节点，建议追问探针。

访谈后分析

每场对话之后，用 Claude debrief：把笔记喂给它，请它指出哪些印证了你的假设、哪些挑战了它、哪些是真正出乎意料的。一批访谈做完后，把全部笔记交给 Claude Cowork 综合，让它浮现反复出现的主题、矛盾点、与两方向上最强的信号。然后把综合输出再带回 Claude，请它标出你对数据的解读是否在“模式匹配到你想听的”，而不是“实际在那里的”。

Exercise： 每完成 5 场访谈，让 Claude Cowork 综合笔记产出两份清单：支持假设的证据，与挑战假设的证据。如果第一份显著长于第二份，请 Claude 评估这种不对称是数据本身的事实，还是你希望看到的事实。

外联与排期

用 Claude Cowork 自动化建联系人清单、对外触达、约访谈的运营工作。它会根据你定义的目标画像研究并整理出结构化潜在客户清单与已验证联系方式；再为每个人按角色与场景批量起草个性化触达邮件；当回复来了，它通过 MCP 接入 Gmail 与 Google Calendar，处理排期与日程；之后按你定义的节奏自动写跟进稿（如 Day-7 跟进），并在每一步完成时更新追踪表。

Exercise： 把已验证的访谈目标画像交给 Claude Cowork，让它建 prospect 列表、起草个性化外联序列、并配套一张追踪表（含外联状态、跟进节奏、访谈完成情况），然后由它跑协调，你专心准备对话本身。

设计你最终的解决方案概念

你已经完成了验证工作：问题是真的，你知道谁有这个问题，你也有一个被证据支撑的方案概念。让 Claude 从每个角度挑战这个方案：哪里有缺口？还有什么替代方案？要在更大规模下成立，需要哪些条件为真？

这是一个重要的现实检查点：这个设计究竟是在解决“验证过程揭示出的那个问题”，而不是你当初臆想的那个？

Exercise： 把方案概念展示给 Claude，让它指出方案最依赖的三个假设；再追问每个假设需要哪些条件为真，以及其中一个不成立时会带来什么后果。

用 Claude Code 搭轻量原型

现在到了有意思的部分：有一个已验证的假设和一个被压力测试过的方案概念，你终于可以“动手”了。

这就是 Idea 阶段里 Claude Code 登场的时刻。即便你之前一直在小幅尝试，从这里起你要正式生成轻量原型：刚好够把你的 idea 摆在一位真实人类面前并获得真实反应的最小表面积。

你不是在做（还不是）面向真实世界的产品；你在搭一个 idea 的功能性样本，用于客户与投资人对话。真实用户对一个能摸到的东西做出的反应，会告诉你 12 场 problem-solution 访谈也说不出事。之前你在确认你解决的问题是真实的；现在你在邀请潜在用户与你提议的解决方案互动。

Exercise： 定义你的方案所依赖的那个最核心的单一交互。让 Claude Code 只造它。做出来之后，把它递给 5 位来自你已验证目标画像的人试用。这 5 段对话里学到的东西，将决定你继续建造还是回到画板。

迈入下一阶段

走到 Idea 阶段终点意味着你在 AI 创业赛跑中一次大跨步：你不再是在赌直觉，而是在依据证据执行。

接下来是 MVP 阶段——创始人的指南问题从“这个值得 Build 吗？”变成“我们应该先 Build 什么？”——AI 的主要角色也从研究伙伴转向施工队。

Chapter 4

MVP 阶段 (最小可行产品)

MVP 阶段：最小可行产品

许多创始人把 MVP 阶段当成“建造期”，但 MVP 阶段本质上仍是证据收集练习。差别在于：你现在收集的证据是关于解决方案的，而非问题空间——具体而言：是否存在一个真实可识别的人群，认为它有用到愿意使用、愿意回来用、愿意付费、愿意告诉他人。

MVP 阶段目标

作为 AI 原生创业公司的创始人，你的目标是把已验证的问题转化为真实用户会真正使用的可工作产品。这不是写满整个路线图的完整版本，而是把真实方案放到真实用户面前、产出 PMF 真实证据的最小、最聚焦的版本。

同时，你现在怎么造，决定了将来能做什么。MVP 阶段还有一个同样重要的第二目标：在不积累“复利型技术债”的前提下快速推进——那种债务一旦真实用户大量到来就会立刻反咬一口。

最后，从第一天开始投资“持久上下文”，才能让 AI 是力量倍增器而非熵的来源。在 AI 原生公司里，代码库是你与 AI 一次又一次协作的对象，所以可读性是基础设施。跳过 spec、架构决策与上下文文件（如 CLAUDE.md）的创始人会撞上一堵可预见的墙：每次新会话都要重新解释代码库，AI 生成的更改也会偏离原始愿景。

MVP 阶段退出标准

退出条件是 PMF 的真实证据：一个具体可识别的用户群体已经认为产品有用到愿意回来用（留存）、付费（收入）、或者告诉他人（推荐）。

MVP 阶段挑战

本阶段创始人的核心指令是速度与判断力。挑战集中在：你能否以正确的方式、用足够快的节奏造出正确的东西，而不留下日后会付代价的捷径。

Agentic 技术债

挑战：AI 几乎抹去了控制“什么东西能进生产”的所有天然瓶颈，速度因此被保证。但当速度成为唯一变量，债务就会膨胀到难以偿还。

一些技术债在 MVP 阶段是合理的，前提是在规模化之前处理掉。它会逐步积累，并可以在一个专项 sprint 里清理。AI 技术债不同：它会复利。没有在 AI 可读位置写下 spec 与架构约束，每次会话都会从零重新推导基础决策，并相互漂移。最终你会得到一个没有连贯心智模型的代码库——不是某一处糟糕，而是它们本就不是设计来组合在一起的。这往往要到很晚才浮出来。

误把 traction 当 PMF

挑战：AI 工具能制造亮眼的早期数字，但这些并不代表市场需要你的产品。

早期势能是创始人最容易被打动的体验。经过数周或数月验证与精进，把产品发出来感觉像“我一直是对的”的确认。

Agentic coding 让你能比以往更快走到这一刻，但早期 traction 不等于 PMF。Launch 时的能量来源于易逝的力量：朋友圈、投资人投资组合里另一家公司的潜在买家、或者一条 Hacker News 标题带来的尖峰流量。这些都不能可靠预测第 6 或第 12 周后会发生什么。

零摩擦 Scope Creep

挑战：当建造毫不费力又近乎免费，总是有“再加一个酷功能”或“再处理一个边界情况”。这种 scope creep 弊大于利。

Scope creep 一直是风险。区别是：以往制衡它的力量——真实工程时间的代价——在加一个特性只需一个下午时不再存在。

棘手之处在于每一次新增都看上去理所应当：产品当然该处理那个边界情况，用户当然会想要那个流程。在 agentic coding 下，单次成本太低，让人感受不到这是 scope creep；但产品越过原始边界扩散开来，你会失去方向和势能。

解药是一份在动手之前写下的范围定义：明确产品做什么、刻意不做什么、以及“真实用户提供的什么样的具体证据”才足以新增功能。把决策点从“我们应该做这个吗？”换成“已经有一批用户告诉我们，没有这个就拿不到价值”。

因为无经验而不安全

挑战：用 AI 工具把应用推向市场而还没有先理解基础安全原则的创始人，最终会让用户暴露在本可避免的风险里。

残酷的现实是：agentic coding 工具生成的是能工作的代码，不是天然安全的代码。可工作的代码容易判断：要么功能跑通要么没跑通。但安全漏洞在被利用之前不可见——这意味着没有自然反馈环来提醒一位首次创始人“出了问题”。把 MVP 上线给真实用户，意味着真实数据、真实暴露面与真实后果。

轻视安全并不是 AI 原生项目独有的。每个时代的自筹创业都常常把安全考虑拖到很晚，甚至到上线前夕。无论何时，在用户接触你的应用之前先做一次安全审查，是最低限度的负责任阈值。

Claude 如何帮助 MVP 阶段创始人

Build 之前先定义架构

在 Claude Code 写第一行生产代码之前，用 Claude 定义并记录架构决策：要遵循的模式、要避免的依赖、做了哪些权衡以及为什么。这份输出将作为聚焦的架构上下文文档，并设定 Claude Code 接下来要在其中工作的护栏。

缺少这份上下文，每次会话都从零开始，Claude Code 被迫自行推断结构性假设——产生的代码库可能能跑，但结构不连贯。在不连贯代码库上迭代与扩张，本质是浪费时间与 token。迟早会有一刻代码不得不重建。

Exercise： 在打开 Claude Code 之前先打开 Claude，向它描述你要造什么：要解决的核心问题、要服务的用户、以及未来 6 个月你能现实预期的规模。请它帮你定义应该贯穿整个 MVP 构建的架构原则、在你这种约束下要避免的依赖、以及你在本阶段有意识接受的权衡。然后把这份输出保存为 CLAUDE.md 文件——你 build 出的第一份产物，也是后续每次会话都要依赖的那一份。CLAUDE.md 是 Claude Code 的项目级指令，Agent SDK 在该目录中运行时会自动读取。功能上它就是你项目的持久“记忆”。

定义并执行 MVP 范围

无摩擦的 scope creep 是 AI 时代 MVP 的典型失败模式之一。就像你为产品定义并记录了应用架构那样，你也必须在动手前定义好 MVP 范围。

用 Claude 起草一份范围文档：MVP 做什么、刻意不做什么、以及新增条款——哪种来自真实用户的具体证据，能在当前阶段证明值得加入新功能。

新功能想法迟早会出现——而它们一定会。届时用 Claude 压力测试它是真实用户信号，还是装扮成产品思考的创始人热情。

用 Claude Code 构建 MVP

架构与范围一旦定义，Claude Code 即成为 MVP 的主要构建工具。用它生成、测试、调试、迭代代码库，但把每次会话视为“执行你已做出的产品决策”，而不是在其中再做新的决策。

每次 Claude Code 会话开始时：(1) 回顾你的范围文档，(2) 把 CLAUDE.md 架构上下文文档提供给模型。会话结束时把会话中浮现的决策回填进去。目标是一个你能说清结构的代码库，而不仅仅是“能跑”的代码库。

Exercise： 为你的 Claude Code 工作建立一个简单的会话模板，包含：架构上下文文档、本次会话的具体任务、需要遵循的约束或模式。会话结束时向上下文文档追加一条简短日志：本次造了什么、做了哪些决策、引入了哪些假设。每次 5 分钟的文档化，是对抗架构漂移的廉价保险。

用户接触前先做安全审查

作为 AI 原生创业公司创始人，你有责任了解代码库里有什么、理解潜在暴露面、不要把明显漏洞推给把数据托付给你的真实用户。

Claude 能为 AI 生成的代码做一次有用的初轮安全审查，识别常见漏洞，是值得加入流程的好习惯。但它不能替代专门的安全工具，也不能替代更高风险场景下的人工审查——把它当作替代品的创始人，最终会成为泄露报道的主角。

Claude Code Security 更进一步：扫描代码库找漏洞，并建议有针对性的补丁供人工审查，能浮现传统方法常错过的问题。

Note： 本手册出版时，Claude Code Security 处于受限 Beta，纳入工作流前请先确认可用性。

Exercise： 部署给任何真实用户之前，把核心应用代码交给 Claude，附上明确简报：审查认证与会话处理、API 响应中的数据暴露、输入校验与注入风险、已知漏洞的依赖。认真对待每一项发现并评估是否需要修复，凡涉及认证、密钥与数据处理的，都要人工审查。

上线前搭好你的度量框架

把早期 traction 误当 PMF 的创始人，通常也是上线后才开始追踪数据的那一批——并且选的指标用来评估“什么在 work”，而不是浮现“什么没在 work”。解药是在第一位用户出现之前就把度量框架建好。

用 Claude 定义对你的产品而言哪些指标真的重要、benchmark 是什么、什么样的数据形态才构成“真实的 PMF”而非好看的噪声。具体地，在发布 MVP 之前就设好留存基线、激活标准、Day-7 与 Day-30 目标。

其次，定义对你这款具体产品而言什么样是假阳性：例如有注册但无激活、有收入但无留存、有初始热情但无重复使用。当数据到来时，让 Claude 对自家 traction 提出反方论证：怀疑者会怎么看这些数字？

管理探索与用户反馈的运营

真实用户进入产品后，运营层会迅速扩张。Claude Cowork 处理那些重要但繁琐的工作：维护用户联系人列表、运行触达序列、安排反馈会、分诊 bug 报告、追踪迭代周期。Idea 阶段管理探索的同一批 MCP 集成在这里同样适用。

在收集反馈这件事里要保留“人在环”：一位用户说“这很好，但我希望它还能……”，需要解读：这是核心需求还是 nice-to-have？是这位客户的特例，还是某一群体的代表？缺失的功能才是问题，还是上游的 onboarding 才是真正的问题？没有工具能替你回答。

Exercise： 配置 Claude Cowork 跑 MVP 阶段反馈回路：起草对早期用户清单的触达、安排反馈会、为 bug 报告与功能请求设计结构化收集流程、并每周综合一份摘要。摘要你先自己读一遍；之后再让 Claude 分析，捕捉任何被忽略的重要点。

为证据而迭代，而不是为完成度

MVP 阶段在你拿到 PMF 真实证据时结束——无论产品看上去多“完成”。宣告“我们达成 PMF”是创始人直觉与已收集证据综合判断的练习。一些有用的试纸：

- Sean Ellis 测试：问活跃用户“如果再也不能用这个产品，你的感受是？”。超过 40% 答“非常失望”，是有意义的 PMF 信号。
- 努力测试：未达 PMF 时，留存靠你不断介入：频繁触达、激励、亲自跟进、英雄式投入。达成 PMF 后，产品开始自己做这件事。当“推”变成“拉”，是最清晰的信号之一。

没有任何单一数据点能确认 PMF：它是一个必须横跨多次迭代仍然成立的模式。

证据要求时就 pivot

投入这么多工作还是无法达到 PMF？结果不印证你最初的方向并非失败，而是系统在工作：MVP 阶段就是为了在你过度投入到错误答案之前让这条信息浮现。

- 探索其他客户细分。也许那些没在转化的用户从一开始就不是合适目标。正确的受众往往已在数据里，只是被低估。
- 调整价值主张。也许受众是对的，但你的 MVP 没和用户产生共鸣。调整 onboarding、信息架构或核心特性侧重，可能不必动既有代码就能解决。

也要保持开放：脱节可能深到需要更根本的调整。

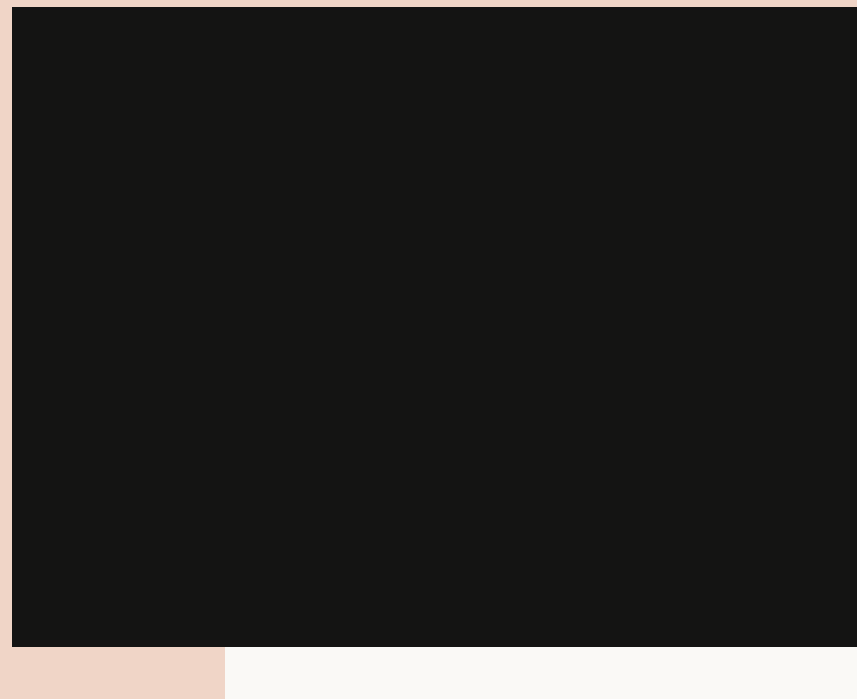
Exercise：如果你已完成 3 次以上迭代但向 PMF 基线没有实质性推进，先让 Claude 做一次诊断再决定下一步。把留存数据、用户反馈与最初的问题假设交给它，问三个问题：

- 数据里是否有某个细分群体的响应明显不同于其他人？
- 设计价值与实际体验价值之间的差，是定位问题还是产品问题？
- 当前产品要达到真正的 PMF，需要哪些条件为真？结合你看到的迹象，这种情境是否现实？

让这些答案决定你要调整、pivot，还是退回到 Idea 阶段。

Chapter 5

Launch 阶段 (上线期)



Launch 阶段：上线期

如果 MVP 阶段证明产品值得存在，Launch 阶段则要证明这门生意值得增长。

Launch 阶段目标

Launch 阶段，创始人必须把早期 traction 转化为可重复、可持续的增长引擎。除了让产品达到 production-ready，你还要在产品下方加固基础设施，同时围绕产品搭建一家真正的公司。

Idea 与 MVP 阶段天然以创始人中心，因为你需要全景态势感知与紧密反馈环。但到 Launch，仍然亲自把每条线拽在手里的创始人会成为瓶颈。目标不是把自己从公司里抽出来，而是搭建运营系统，把你的注意力解放出来去做只有创始人能做的决策。

Launch 阶段退出标准

1. 增长可重复、由渠道驱动：你不仅在留存用户，而且能通过特定渠道以已知单位经济学（CAC、LTV、回本周期）可预测地获取用户。
2. 产品能承受生产工作负载：基础设施加固完毕，安全与合规到位，在真实生产条件（而不仅仅是你测试过的条件）下保持稳定。
3. 运营能在无创始人瓶颈下运转：流程已存在，自动化已落地。你不再亲自处理支持、分诊、sprint 规划或报告。

Launch 阶段挑战

找到 PMF 是早期生命周期里最难的问题。现在的挑战是保住它。如果围绕产品的组织跟不上，已经获得 PMF 的公司仍可能在这里散架。

技术债到期

挑战：为速度与验证而搭的 MVP 代码库足以证明产品可用，但生产流量、新功能与日益增长的复杂度，正在把那些捷径暴露出来。

在 MVP 阶段积累一些技术债是合理权衡。Launch 阶段开始计息，越拖越贵。解法包括：系统化架构审计找出结构性弱点，针对最严重处的有针对性重构，以及实质性的测试覆盖扩展，让下一波功能不再带来旧问题。

创始人成为瓶颈

挑战：在 MVP，创始人事事在环是资产。在 Launch，支持量增长、产品决策堆积、运营复杂度倍增——这种本能变成了约束。

从亲手做事到设计“做事的系统”——这是创业生命周期里最难转变之一。因为转变发生得没有清晰瞬间，风险在于完全错过它，组织停滞而你还在 builder 模式。征兆包括：本该一小时的决策现在要一周；只有你知道答案的支持请求开始堆积；只有当你亲自记起时才会发生的运营任务越来越多。

解药是对你亲自承担的事做一次彻底审计：从最琐碎的小任务到最高风险的决策，识别哪些可以系统化、哪些可以委派、哪些真的值得创始人时间。

安全与合规不再可推迟

挑战：MVP 阶段保持简单的安全与合规没问题；现在面对真实用户、真实数据与可能在桌上的企业合同，它会变成负债。

MVP 阶段、少量 beta 用户、无敏感数据时，安全漏洞是理论风险。产品进入真实用户依赖的生产那一刻，假设变成真实暴露。原先不适用于原型的合规要求，一旦你开始处理客户数据、收款或销售给受监管行业，立刻适用。

解药：在规模到来之前而不是之后做系统性安全与合规审查；把所有浮现的问题视为下一波用户到来前必须解决的整改项，而不是建议。

尚未准备好就扩张

挑战：新市场与融资机会看起来都是增长机会，它们也可能是 PMF 死亡的地方。

你的初始 traction 是真实的，但也对早期受众有特异性。过早扩张到一个与原始市场显著不同的新市场，会引入你的产品并未为之设计的用户行为、合规要求、支付基础设施与基线期望。新变量太多，你失去清晰解读自家数据的能力，还可能在追逐新且未经验证的受众时忽视原始用户群。

Claude 如何帮助 Launch 阶段

三种 Claude 形态在 Launch 阶段全面投入使用，并互相支撑：每种工具的输出成为另两种的输入。结果自然复利——同时使用三者的创始人，得到的远超工具之和。

这正是“超精益创业模型”在结构上可行的原因：Claude Code 造产品，Claude Cowork 围绕产品造公司，Claude 把产品与组织知识运营化——一支小团队可以像数倍规模的公司一样运作。

在债务复利之前处理掉技术债

MVP 代码库能跑，但需要系统化整改，找出可能成为结构性负债的技术债。先用 Claude Code 做一次完整架构审计：哪里脆弱，哪些捷径将来维护代价高，哪里测试覆盖薄到下一波功能会再次引入相同问题。

把 Claude Code 的审计结果交回 Claude，做分诊与排序：哪些必须在下个 release 前修，哪些可以等一个 sprint，哪些是当前阶段可接受的持续债务。

这也是把 MVP 时期那些“只在你脑子里”的架构决策落地的时刻。把它们写进 CLAUDE.md，确保未来每一次 Claude Code 会话都对“系统如何被设计以及为什么这样设计”的共同理解开始。

Exercise：让 Claude Code 审计你的 MVP 代码库，给出按优先级排序的结构弱点、测试覆盖缺口与重构候选清单。把这份清单交给 Claude，按多个 sprint 排序整改工作：必须先做的关键事项、可与功能开发并行处理的、可后置的。

搭建能替代创始人注意力的系统

要构建释放注意力、让创始人专注于“只有创始人能做的事”的运营系统，首先要弄清自己的注意力实际去哪儿了。让 Claude Cowork 做一次结构化的当前运营负荷审计：每一项重复任务、每一项落到你桌上的决策、每一条只因你亲自记起才会发生的工作流。然后让它把库存分类：哪些可完全自动化、哪些需要人但不必是你、哪些真的需要创始人判断。

审计完成后，用 Claude Cowork 为自动化候选设计工作流逻辑：每个工作流由什么触发、决策规则是什么、输出是什么、完成后流向哪里。

把安全与合规变成一条产品 workflow

用 Claude Code 浮现 SOC 2、GDPR、HIPAA 审计或目标市场要求的标准里常见的代码级问题。这会同时暴露漏洞与合规缺口。把这些发现交给 Claude，帮你为整改排序，并设计企业买家签约前会要求的控制、审计日志与访问管理。注意：AI 扫描是辅助，不能替代有资质的合规审查。

接下来，把合规工作纳入你的开发节奏，而不是一次性项目；合规文档需要持续维护与更新。如果你正在接近企业合同或国际市场，这也是 Claude Code 安全扫描帮你准备独立安全评估的时机。

Exercise：用 Claude Code 按目标市场所要求的框架做一次代码级安全审查。把输出交给 Claude，让它产出两份东西：按优先级排序的安全整改序列；以及向企业买家通过合规审查所需的文档与控制清单。

把你之前跳过的 PM 流程立起来

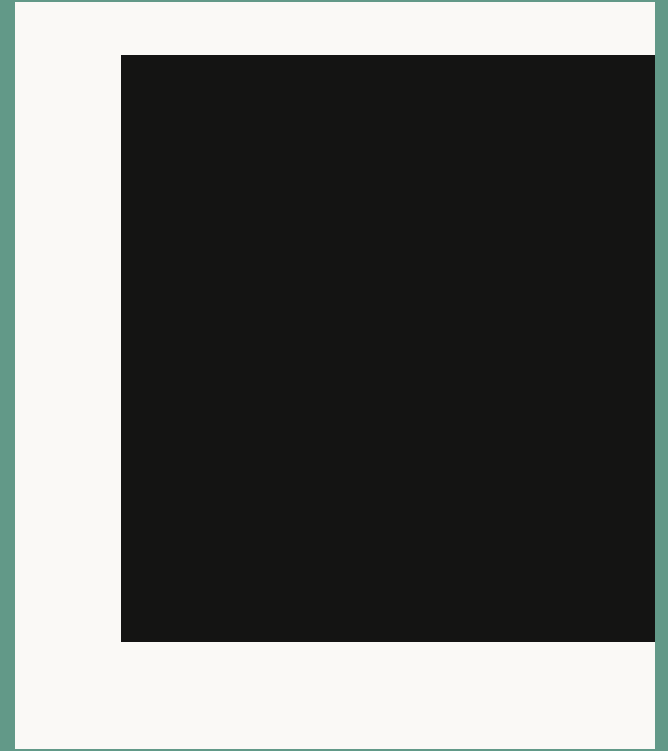
Launch 阶段需要一套轻量、可重复、不靠创始人触发也能跑的流程。用 Claude 设计产品时间线与工作节奏：在 Claude Code 动手某个功能之前 spec 应包含什么；bug 报告如何分诊与路由；每周指标报告覆盖什么、如何分发。

流程设计完成后，用 Claude Cowork 搭建并运营这一层：安排 sprint 仪式、把接入的 bug 报告路由到对应位置、从已连接的数据源汇编每周指标、维护用户信号流入产品决策的反馈环。

Exercise：请 Claude 设计一套轻量 PM 操作系统：确定的 sprint 节奏、最小 spec 模板、bug 分诊决策树、以及一份从真实数据源拉取的周报。然后设置 Claude Cowork 实现并按计划运行其中重复性运营要素。

Chapter 6

Scale 阶段 (扩张期)



Scale 阶段：扩张期

Scale 阶段，创始人的角色从 builder 重新归位为面对外部世界的 executive。产品仍然是核心，但你日常工作越来越多关于公司本身。在保持精益、AI 中心化的结构优势同时，你的注意力必须扩展到 Scale 阶段的新活动：分析师沟通、IPO 路演等。

Scale 阶段目标

扩展技术基础设施的工作还在继续，现在还要加上扩展组织本身、把它打磨成一家成熟生意的工作。

在此阶段你要把用户从几千扩到几百万、从一个市场扩到多个市场。在之前每个阶段，你都能凭借贴近用户、依据紧凑反馈环加上一份创始人直觉来感知增长方向。现在，目标是用成熟的组织运营来支撑系统性增长。

对 AI 原生创业公司而言，目标是通过“积累式深度”构筑可防御的护城河：你的产品中沉淀的领域专长、产品与用户依赖工具 / 平台的集成深度、独有的系统数据与工作流。一直在同一方向、同一基础设施上持续构建的创始人，此刻拥有了真正难以复制的东西。

此阶段，公开投资者、分析师、监管者、企业采购团队与潜在收购方会带来更大压力——以及更高怀疑——因为代价更高。你的产品与组织必须能经受外部审视：不仅是你造了什么的能力，还有围绕它的治理、合规姿态、财务控制与战略叙事。

Scale 阶段退出标准

Scale 阶段的退出条件不再是单一里程碑，而是一个临界事件：哪怕创始人越来越不直接掌管日常运营，公司仍可持续。你已展示出系统性增长，建立了能满足最苛刻外部评审的组织治理与合规基础设施，并能稳妥回答：“如果一家资金充裕的在位者今天复制了你的产品，你的用户会留下吗？”

实践中，这个阈值通常呈现为三种之一：在不再需要外部资本的规模上可持续盈利、IPO-ready、或被收购。三者都要求增长是系统化、可审计的，护城河经得起审视，组织运营成熟且可持续。

做到这一点，恭喜：你的 startup 从一次下注变成了一门生意。

Scale 阶段挑战

把运营层授权出去

挑战：Scale 阶段的运营系统必须无需“看管”也能可靠运转。对从 day one 亲力亲为的创始人，这与其说是结构挑战，不如说是心理挑战。

Launch 阶段你做的是“创造系统”；Scale 阶段则变成：(1) 把这些系统打磨到完全可信，(2) 然后真的去信任它们。

这比听起来难。即便是擅长授权的创始人，也并不总是清楚什么该放手、什么该留下。放得太多太快——尤其放给 AI 自动化系统——关键决策可能在缺少只有创始人能提供的关键上下文时做出。留得太久，又会成为瓶颈。

根本挑战是：识别那些只存在于创始人脑中或未文档化的 workflows 里的机构性知识，把它编码进可文档化、可审计、可移交的系统。

技术运营扩张

挑战：客户评估的不再只是你的产品；他们要知道你的组织能不能做可靠的基础设施伙伴。

前三个阶段，技术挑战集中在代码库本身：在不积累技术债的前提下造对的方案，然后为真实用户加固安全与合规。Scale 阶段挑战变成围绕代码库的一切：搭建支持基础设施、文档、可靠性保证，传递成熟度信号。

更大规模的客户、签多年合同的机构采购方，会在签约前要求这些东西，并在签后据此考核你。让你走到这里的同一套 AI 基础设施，能帮你建立带有定义响应时间的专门支持职能，以及新客户工程团队真的能用的文档。

组织职能扩张

挑战：Scale 阶段公司无论由多少人在运营，都需要招聘、薪酬、会计、法务运营等组织基础设施。

在 Launch，“系统化运营”意味着把消耗创始人注意力的工作流自动化。Scale 阶段需要扩展一类更广、某种意义上更要紧的运营职能：财务报告、合规监控、合同管理、客户成功等。

搭建一个真正的 GTM 职能

挑战：有机增长有上限，多数 Scale 阶段创始人在构建真正的 GTM 职能之前就先撞到这上限。

Idea、MVP、Launch 阶段的增长常来自创始人亲自带的销售：一次掐好时机的 Product Hunt 帖子、早期客户的私人关系。这类有机增长走到一定程度就到顶——多数 startup 在 Scale 阶段触顶。征兆：用户曲线变平、获客成本上升、pipeline 只在创始人亲自参与时才推进。

Scale 阶段增长需要专门的增长引擎触达更新更广的受众。多数创始人此前没运营过 marketing、销售、分析师关系。正经 GTM 不仅要新系统流程，还要为你想如何谈论产品打造品牌声音与故事。

好消息是：GTM 不必很大也能有效，让你造出产品的同一 AI 基础设施，也能把它带到市场。

Claude 如何帮助 Scale 阶段

前面几个阶段，Claude 作为产品本身的基础设施：验证 idea 的研究伙伴、设计与搭建原型的工程团队、让单人 startup 成为可能的 AI 运营层。走到 Scale 阶段的 AI 原生创业公司，可以继续用 Claude、Claude Code 与 Claude Cowork 以同样方式扩展业务。

把日常事务交给 Claude Cowork

在 Scale 阶段开始时，先清晰地看清你现在最需要把时间与注意力投到哪里。让 Claude 帮你列出“只有你应该做的事”清单：产品叙事决策、董事关系、企业级 deal、创始人对创始人的对话。不在清单上的，都可委派或由 Claude Cowork 自动化。

Exercise： 用 Claude 画一张当前运营层的瓶颈地图：每一项当前要经过你的工作流、决策与审批。然后让 Claude 推演：如果你一周不可用，每一项会发生什么？停滞的那些就是你仍然亲手把着的环节。

把它们与你和 Claude 一起整理的创始人优先级清单做对照。接下来，压力测试已经搭起的系统是否真的随业务扩展而扩展。

Exercise： 用 Claude 画出当前工作流，再问它在你一周不可用时各项会怎样。停滞的那些，是交接标准、升级路径或异常处理仍需收紧的地方。Claude 可以分析失败点并建议合适修复，你据此更新或替换相应 Claude Cowork 自动化。

把技术运营扩展为企业级基础设施

随着规模扩大，买家需要被打消顾虑：你的产品与组织能作为长期基础设施被信任。代码内的技术工作如常进行，但围绕代码库的技术工作也要做了。

第一步是把机构知识转成可扩展的系统。用 Claude 起草并维护企业采购期望看到的书面材料：产品文档、支持 playbook、SLA。

并行地，让 Claude Code 按企业合同所要求的可靠性与安全标准审计并加固代码库，搭建过去 Discord 社区支持时无需提供的技术支持基础设施：日志、监控、事件响应工具、让 SLA 真正可执行的可观测层。

Claude Cowork 再把企业支持的运营层跑起来：工单路由、升级流程、由产品变更触发的文档更新、续约追踪、企业客户成功依赖的报告节奏。三者合起来，让一支小团队呈现远比规模更大的组织的支持姿态——这恰是签下多年期企业合同所需要证明的。

Exercise： 挑出你最难搞的三个潜在客户，或你最想签下的三个理想客户。请 Claude 做一次 gap 分析：他们的企业采购团队在签多年期合同前会期待看到的文档、SLA 与支持基础设施分别是什么，你目前在哪里有缺口？据此把 Claude Code 与 Claude Cowork 的技术与文档工作排序。

建立一个真正的 GTM 职能

创始人的“肝”把你带到了这里，但扩张需要真正的 go-to-market 战略。AI 能帮你搭建并运营一整套 GTM 引擎。

Claude 可以从零搭建 GTM 基础资源：市场细分、信息架构、分析师关系策略、销售 playbook、面向公开市场投资者、企业买家与华尔街分析师所看重的指标叙事。每一类受众都有自己的词汇与评判标准；Claude 的工作是把你的产品价值主张翻译成对每一类受众都相关的产品营销路径。

Claude Cowork 则成为你的战术执行层：内容流水线、外发序列、分析师简报后勤、媒体与 PR 节奏、CRM 卫生、pipeline 报告，以及把 GTM 战略转为真正商业动作的诸多重复周期。

如果 GTM 需要产品营销基础设施——交互式 demo 环境、集成文档、沙盒租户、API 参考、技术 one-pager——Claude Code 可以为你搭建。买家期望从技术角度评估你的产品；在 Scale 阶段，一段 Loom + 一份销售 PPT 不够。这也是让 GTM 异步运作的基础设施：当你在董事会上时，一份做得好的 demo 环境正在帮你成交。

把领域专长沉淀为 AI 上下文

许多超精益创业的创始人，是在为他们亲身体会或观察到的某个行业问题做高度具体的应用或工具。Agentic AI 让从未写过一行代码的创始人也能用领域专长造出能解决精细问题的产品。Claude、Claude Code 与 Claude Cowork 各自在“把创始人知识转化为复利型产品特异性”中扮演角色。

用 Claude 捕获、组织、提炼创始人知识，把领域专长放到产品可以触达的地方。通过长对话、project 与 memory，创始人可以把行业黑话、监管陷阱、边界情形、用户挫败、为什么“显而易见的答案”在此行不通——全部沉淀为结构化、可检索的上下文。Skill 进一步把高频工作流（例如“我审查商业租赁合同的方法”、“我分诊一份病人初诊表单的方法”）编码成可复用例程，Claude 每次都以同样方式执行。几个月后，这变成通用 AI 永远赶不上的专有知识层。

Claude Code 能把你这个行业里其他从业者的常见挫败翻译为校验逻辑、Prompt 调整或对你竞争对手都没听说过的小众行业系统的 MCP 集成。这样，你的产品的深度与广度都会以竞争对手难以复制的方式持续复利。

Exercise： 在你的垂直里挑一个通用竞品一定会做错的边界情况。与 Claude Code 一起，基于你真正见过的场景为它写一个专门的测试用例（不是单元测试）。每出现一个相似边界情况就加一条。你的测试集会变成你护城河的地图。

把累计的用户数据复利成可防御优势

用户在产品里产生行为信号（接受什么输出、拒绝什么输出），它会反过来塑造路线图。日积月累，你会学到属于自己用户群的具体模式、偏好与边界情形。这就是我们所说的复利价值：每次改进让产品更有用、带来更多使用、产生更多反馈、驱动下一次改进。

这些数据时间锁定、上下文特定，山寨者无法复制：你买不到成千上万用户在你产品里打磨 workflow 所形成的行为指纹。

让 Claude 审计你已收集的用户交互数据、识别其中信号最强的行为模式、设计把持续使用变成系统性模型改进的反馈环。

Exercise： 把你产品交互数据的概要喂给 Claude：你在收集什么、收集多久了、你了解到的用户随时间的使用方式。请它识别其中三个信号最强的行为模式，并为每一个设计一条把它转化为系统性模型改进的反馈环。然后请它帮你起草一页“护城河叙事”用于产品营销：你的数据飞轮如何运转、它已转了多久、为什么一个今天才入场、资金充裕的竞争者也无法在两年内复制。

建立 workflow 锁定

复利的数据网络效应让你的产品更难被复制；而用户 workflow 的锁定让你的产品更难被离开。用户在日常运营里运行你产品的时间越长，它就越深地嵌进他们工作的方式。他们已在其上搭建自动化、培训了团队、连接了数据源与其他工具。他们打磨的提示、改良的 workflow、标准化的输出，都围绕你产品的做事方式生长。到这一步，“切换”从产品决策变成全规模运营项目。

建立 workflow 锁定的第一步：让 Claude 按集成深度盘点你当前的客户群。对每个客户细分，识别他们在你产品上搭建了哪些 workflow、依赖哪些 integrations。这显示你的产品在哪里粘住了，哪里需要更深。

你提供的 integrations 越多，客户构建依赖你产品的工作流的表面积就越大。Claude Code 能快速搭出与目标用户依赖的数据管道、项目管理工具与其他系统的原生集成。Claude Code 还能造出 API、Webhook、SDK，让客户不仅使用你的产品，还能在其上构建——这是最深层的锁定。

Exercise： 请 Claude 帮你为前十大客户做一次 workflow 集成审计：记录每位客户搭建的自动化、依赖的集成、跑在你产品上的团队 workflow，以及你对其切换成本的估算。然后让 Claude 识别群组层面的模式：对你的具体产品而言哪种类型的集成带来最深锁定？还可以做什么或开放什么，以加深当前仅在表面集成的客户的绑定？

Chapter 7

同样的工作， 新的规则



同样的工作，新的规则

在 AI 时代，创始人的工作并没有变：找到一个真实问题，造出能解决它的东西，把它扩展成一家有分量的公司。变的，是抵达那里的路径。在 Idea、MVP、Launch、Scale 四个阶段之间，AI 把以季度计的工作压缩成以周计。

曾经要数月的验证周期，如今一个下午就能完成。能跑的原型不再需要一位有合适技术栈的联创——它只需要一个清晰的问题与几次和 coding agent 的聚焦会话。Launch readiness 从发布前的临阵磨枪变成了一条持续运转的工作流。到 Scale 阶段，曾经把早期员工压成救火队员的运营重负，可以越来越多地交给 AI，把团队的注意力释放出来，去做那些最终成为你护城河的判断决策。

瓶颈不再是你能造什么，而是你选择造什么。

延伸资源



延伸资源

用 Claude 构建

- Building AI Agents for Startups
如何用 agents 让 startup 在扩张中更少依赖创始人。
- Claude Code docs
从初装到高级 agentic 工作流。建议从 “How Claude Code works” 总览开始。
- Claude Code best practices
Anthropic 内外工程团队验证过的实践——上下文管理、权限、规划与校验工作流。
- Using CLAUDE.md files
为你的具体代码库配置 Claude Code。MVP 阶段搭开发环境的必读。
- Claude Code power user tips
来自 Claude Code 团队自身的工作流模式：并行会话、校验回路等。
- Get started with Claude Cowork
团队如何配置 Claude Cowork，落地 skill、plugin 等放大它价值的能力。
- Tutorials
claude.com/resources/tutorials 提供可检索的实操指南清单。

创始人故事

- How three YC startups built with Claude Code
看 HumanLayer (F24)、Ambral (W25)、Vulcan Technologies (S25) 如何用 Claude 把原型快速推向市场并扩展 AI 平台。
- GC AI
创始人用领域专长造出响应式的、Claude 驱动的法务平台，匹配内部团队真实的工作方式：公司专属 playbook、跨职能 stakeholder、可变风险阈值。
- Carta Healthcare
用 Claude 驱动其临床抽取平台，每年处理 22,000 例手术案例，把数据抽取时间减少 66%。
- Anything
基于 Claude + Agent SDK，已让 150 万用户在不写代码的情况下把想法变成可工作的软件产品。
- Cogent
应用 AI 实验室，用 Claude 作为 agent 推理层，在完整漏洞生命周期内自动化调查、优先级判断与修复。
- Airtree
以 Claude Cowork 为运营核心，统一过去散落在十多种工具与团队中的数据；一人用 skill 搭出的工作流自动化，全员都能复用。

- Duvo
造 AI agent，跨 ERP、供应商门户、表格、邮件甚至电话，运行采购、供应链与品类管理流程。整体基于 Claude，用 Agent SDK 跨工作流编排。
- Zingage
为居家护理机构提供 7x24 自动化运营的 AI agent 平台。用 Claude 的结构化工具调用跨 EMR 与多通信渠道编排，并借 Claude 的上下文推理给出针对患者的细致输出。
- Kindora
由非营利高管搭建的 AI 平台，用 Claude Sonnet 智能匹配慈善机构与资助方。Kindora 的 MCP 连接器让非营利组织直接在 Claude 内访问其线索工具。
- Wordsmith
由“律师转 CTO”创办，为内部法务团队提供可靠的 AI 法律技术。Claude 是 Wordsmith 合同审查、协议起草与文档审查的推理引擎；团队也用 Claude Code 演进平台本身。

创业支持与机会

- Anthropic Startups Program
与 Anthropic VC 合作的 startup 可获得免费 API 额度、公开可用的最高 tier 速率限制，以及独家创始人活动与工作坊邀请。
- Claude community
Builders 的论坛与社区空间。
- Live learning resources
大会、直播、网研、回放等学习资源。

claude.ai